Linux Kernel Programming

Linux Kernel programming

Agenda

- Process Management – CFS
- Process Management – RT
- Process Management – Stop
- Process Management – Deadline
- Process Management – IDLE
- Process Management – Context Switch
- Process Management – process States

Process Management - CFS

Scheduling classes and policies

- Stop
  - No policy
- Deadline
  - SCHED_DEADLINE
- Real Time
  - SCHED_FIFO
  - SCHED_RR
- Fair
  - SCHED_NORMAL
  - SCHED_BATCH
  - SCHED_IDLE
- Idle
  - No policy

Process Management - CFS

Run queues

- The core scheduler manages ready threads by enqueueing them into a run queue, which is implemented by struct rq.

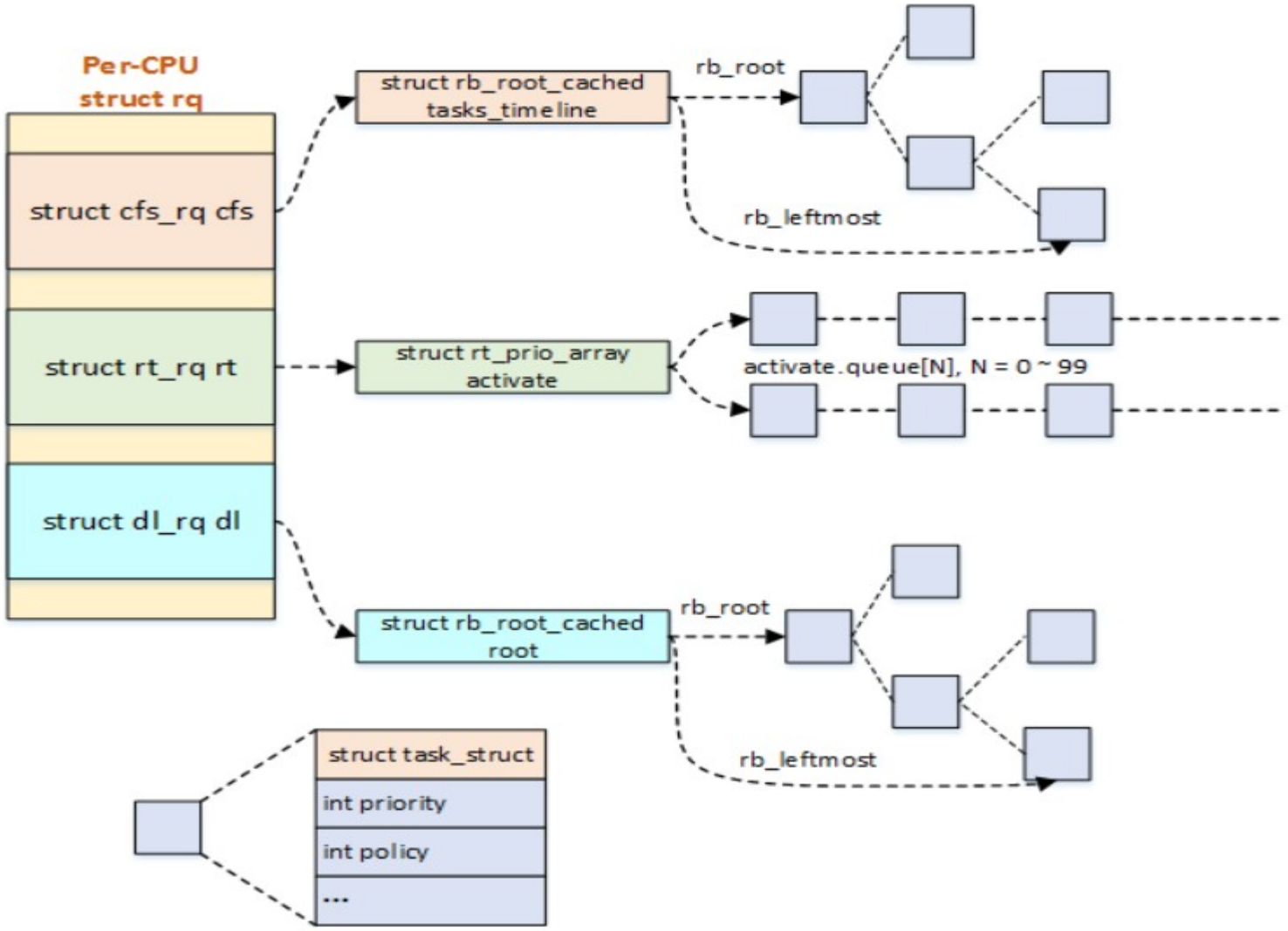
```
struct rq {
    ...
    struct cfs_rq cfs;
    struct rt_rq rt;
    ...
    struct task_struct *curr, *idle, *stop;
    ...
    u64 clock;
    ...
};
```

Process Management - CFS

- Each CPU has its own run queue, i.e., there are as many run queues as the number of CPUs in a system. A ready thread can belong to a single run queue at a time, since it is impossible that multiple CPUs process the same thread simultaneously. Here comes the need of load balancing among CPUs in multi-core systems. Without a special effort to balance load, threads may wait in a specific CPU's run queue, while other CPUs have nothing in their run queue, which, of course, means performance degradation.

- A run queue includes struct cfs_rq cfs and struct rt_rq rt, which are sub-run queues for the CFS and the RT scheduler, respectively. Enqueueing a thread into a run queue eventually means enqueueing it into either of these sub-run queues depending on the scheduler class of the thread. The thread that is currently running on a CPU is pointed by struct task_struct *curr defined in the run queue of the CPU. The per-run queue variable clock is used to store the latest time at which the corresponding CPU reads a clock source.

Process Management - CFS



Process Management - CFS

- The function **__schedule()** is the main body of the core scheduler. What it does includes putting the previously running thread into a run queue, picking a new thread to run next, and lastly switching context between the two threads.

```
static void __sched __schedule(void)
{
    ...
    cpu = smp_processor_id();
    rq = cpu_rq(cpu);
    prev = rq->curr;
    ...
    put_prev_task(rq, prev);
    ...
    next = pick_next_task(rq);
    ...
    if (likely(prev != next)) {
        ...
        rq->curr = next;
        ...
        context_switch(rq, prev, next);
        ...
    }
    ...
}
```

Process Management - CFS

- Most work in __schedule() is delegated to the scheduling classes. For example, when put_prev_task() is invoked in __schedule(), actual work is done by the function registered to the function pointer put_prev_task of the scheduling class that the previously running task belongs to.

```
static void put_prev_task(struct rq *rq, struct task_struct *prev)
{
    ...
    prev->sched_class->put_prev_task(rq, prev);
}
```

Process Management - CFS

- As shown in rt_sched_class and fair_sched_class, this is put_prev_task_rt() for the RT scheduler and put_prev_task_fair() for the CFS. A similar thing applies when pick_next_task() is invoked.

```
#define for_each_class(class) \
   for (class = sched_class_highest; class; class = class->next)

static inline struct task_struct *pick_next_task(struct rq *rq)
{
    const struct sched_class *class;
    struct task_struct *p;
    ...
    for_each_class(class) {
        p = class->pick_next_task(rq);
        if (p)
            return p;
    }
    ...
}
```

Process Management - CFS

- The function pick_next_task() seeks the next-running thread using the function pointer pick_next_task of a scheduling class by which pick_next_task_fair() and pick_next_task_rt() are called for the CFS and the RT scheduler, respectively. Note that by the for_each_class(class) loop, scheduling classes are processed one by one in order of their priority, by which fair_sched_class can be taken care of only if there is nothing to do with the rt_sched_class.

- The function __schedule() is invoked at many places of the kernel, where there is a need to reschedule threads. One of such cases is after interrupt handling, since by an interrupt, some thread (e.g., a high-priority RT thread) may need to run immediately. Another case is when someone calls it explicitly. For example, a system call sched_yield() that causes the calling thread to relinquish the CPU is implemented using __schedule().

Process Management - CFS

- The function scheduler_tick() is periodically called by the kernel with the frequency HZ, which is the tick rate of the system timer defined on system boot.

```
void scheduler_tick(void)
{
    int cpu = smp_processor_id();
    struct rq *rq = cpu_rq(cpu);
    struct task_struct *curr = rq->curr;
    ...
    update_rq_clock(rq);
    curr->sched_class->task_tick(rq, curr, 0);
    ...
}
```

Process Management - CFS

- The first thing among what scheduler_tick() does is updating clocks invoking update_rq_clock(). The update_rq_clock() reads a clock source and updates the clock of the run queue, which the scheduler's time accounting is based on. The second thing is checking if the current thread is running for too long, and if it is, setting a flag that indicates that __schedule() must be called to replace the running task with another. This done by calling task_tick in a scheduler class. Again, actual work is delegated to a scheduler-class-specific function pointed by this function pointer.

Process Management - CFS

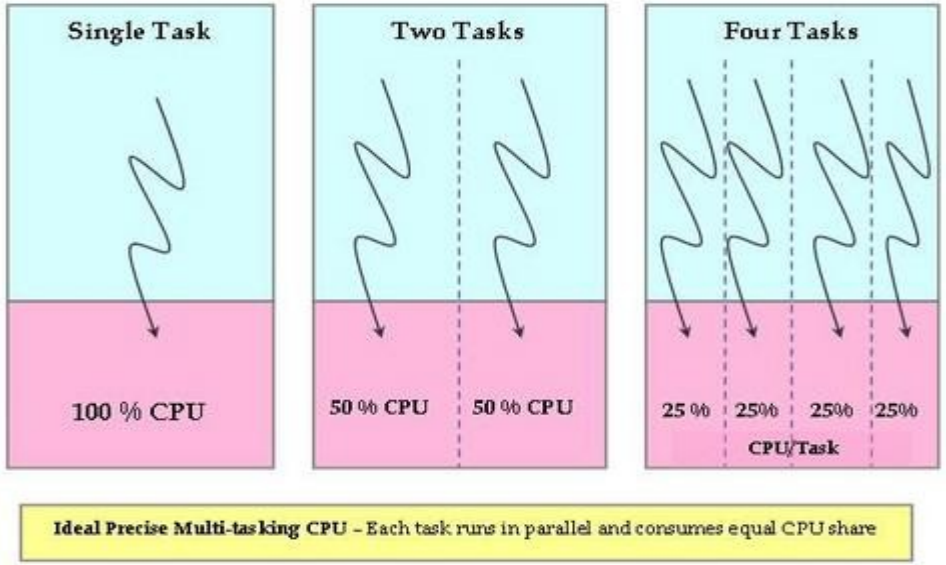
Sched class: CFS (Completely fair scheduler)

- Introduced by Ingo Molnar (motivated by Rotating Staircase Deadline Scheduler by Con Kolivas).
- Scheduling policies:
  - SCHED_NORMAL: Normal Unix tasks
  - SCHED_BATCH: Batch (non-interactive) tasks
  - SCHED_IDLE: Low priority tasks
- Implemented with red-black tree (self balancing).
- Tracks Virtual runtime of tasks (amount of time task has run).
- Tasks with shortest vruntime runs first.
- Priority is used to set task's weight, that affects vruntime.
- Higher the weight, slower will vruntime increase.
- Task's priority is calculated by 120 + nice (-20 to +19).
- Used for all other system tasks, eg: shell.

Process Management - CFS

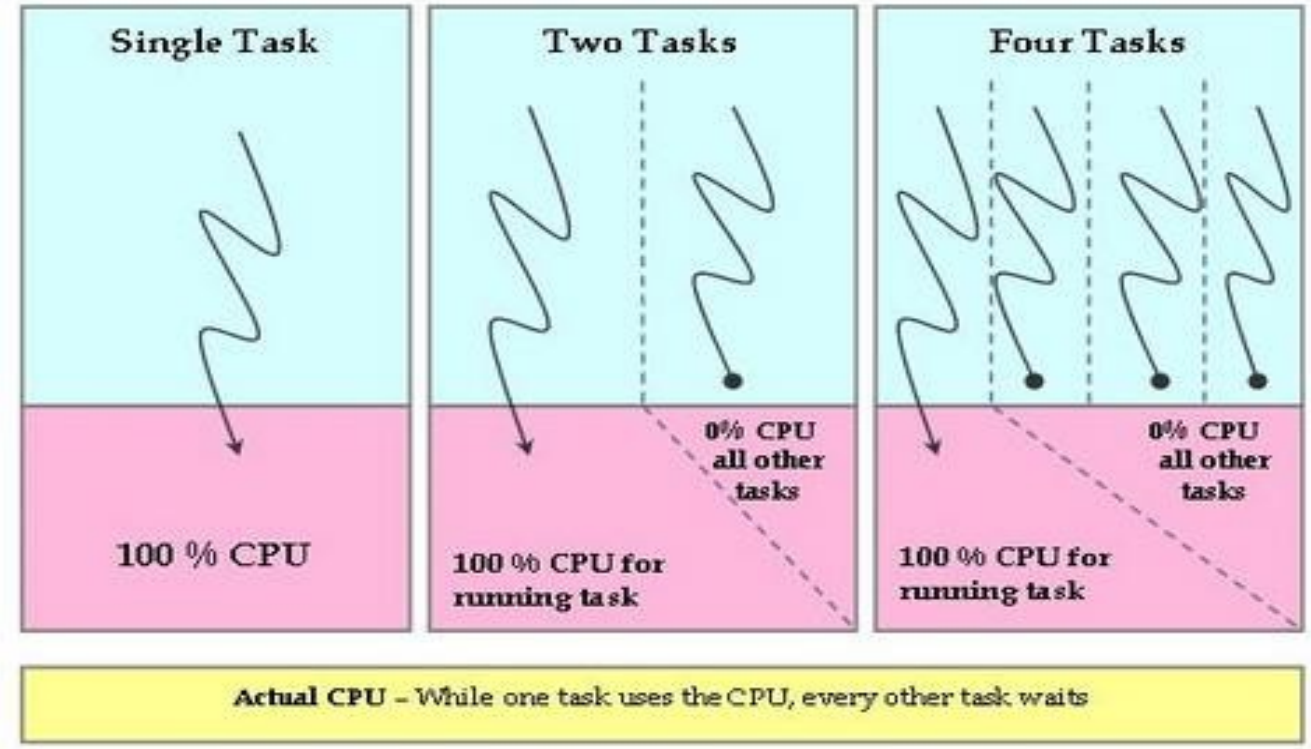
- CFS tries to be “fair” to every task running in the system. Completely Fair Scheduler According to Ingo Molnar, the author of the CFS, its core design can be summed up in single sentence: “CFS basically models an 'ideal, precise multitasking CPU' on real hardware.”

- Let's try to understand what “ideal, precise, multitasking CPU” means, as the CFS tries to emulate this CPU. An “ideal, precise, multitasking CPU” is a hardware CPU that can run multiple processes at the same time (in parallel), giving each process an equal share of processor power (not time, but power). If a single process is running, it would receive 100% of the processor's power. With two processes, each would have exactly 50% of the physical power (in parallel). Similarly, with four processes running, each would get precisely 25% of physical CPU power in parallel and so on. Therefore, this CPU would be “fair” to all the tasks running in the system.



Process Management - CFS

- Obviously, this ideal CPU is nonexistent, but the CFS tries to emulate such a processor in software. On an actual real-world processor, only one task can be allocated to a CPU at a particular time. Therefore, all other tasks wait during this period. So, while the currently running task gets 100% of the CPU power, all other tasks get 0% of the CPU power. This is obviously not fair.



Process Management - CFS

- The CFS tries to eliminate this unfairness from the system. The CFS tries to keep track of the fair share of the CPU that would have been available to each process in the system.

- The main idea behind the CFS is to maintain balance (fairness) in providing processor time to tasks. This means processes should be given a fair amount of the processor. When the time for tasks is out of balance (meaning that one or more tasks are not given a fair amount of time relative to others), then those out-of-balance tasks should be given time to execute.

- To determine the balance, the CFS maintains the amount of time provided to a given task in what's called the virtual runtime. The smaller a task's virtual runtime—meaning the smaller amount of time a task has been permitted access to the processor—the higher its need for the processor. The CFS also includes the concept of sleeper fairness to ensure that tasks that are not currently runnable (for example, waiting for I/O) receive a comparable share of the processor when they eventually need it.

- But rather than maintain the tasks in a run queue, as has been done in prior Linux schedulers, the CFS maintains a time-ordered red-black tree. A red-black tree is a tree with a couple of interesting and useful properties. First, it's self-balancing, which means that no path in the tree will ever be more than twice as long as any other. Second, operations on the tree occur in O(log n) time (where n is the number of nodes in the tree). This means that you can insert or delete a task quickly and efficiently.

Process Management - CFS

- Since version 2.6.23, CFS has been the de facto kernel CPU scheduling code for regular threads; the majority of threads are SCHED_OTHER, which is driven by CFS.

- The driver behind CFS is fairness and overall throughput. In a nutshell, within its implementation, the kernel keeps track of the actual CPU runtime (at nanosecond granularity) of every runnable CFS (SCHED_OTHER) thread; the thread with the smallest runtime is the thread that most deserves to run and will be awarded the processor on the next scheduling switch. Conversely, threads that continually hammer on the processor will accumulate a large amount of runtime and will thus be penalized.

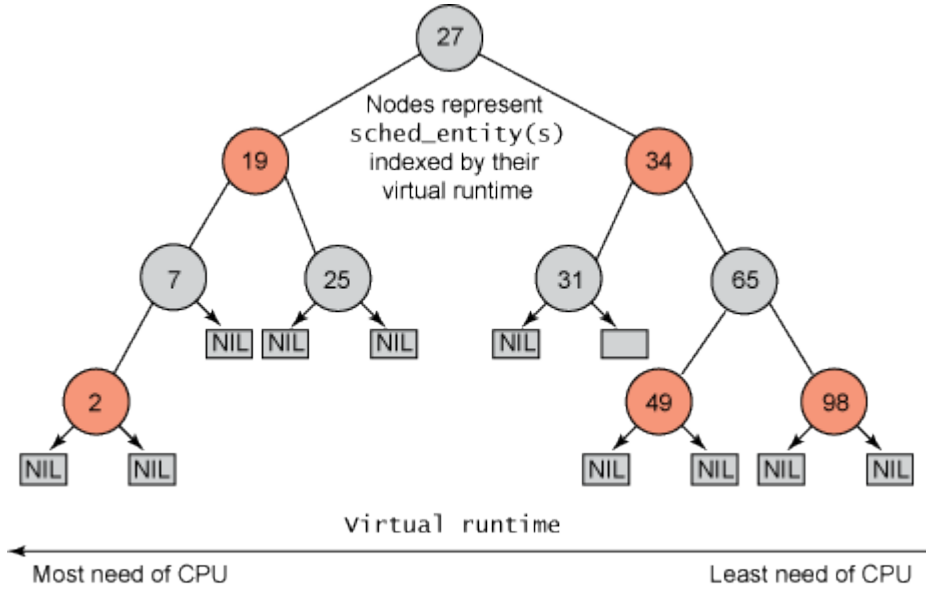
- The internals of the CFS implementation, embedded within the task structure is another data structure, **struct sched_entity**, which contains within it an unsigned 64-bit value called **vruntime**. This is, at a simplistic level, the monotonic counter that keeps track of the amount of time, in nanoseconds, that the thread has accumulated (run) on the processor.

- In practice, here, a lot of code-level tweaks, checks, and balances are required. For example, often, the kernel will reset the vruntime value to 0, triggering another scheduling epoch. Also, there are various tunables under **/proc/sys/kernel/sched_*,** to help better fine-tune the CPU scheduler behavior.

Process Management - CFS

- CFS picks the next task to run is encapsulated in the **kernel/sched/fair.c:pick_next_task_fair()** function. In theory, the way CFS works is simplicity itself: enqueue all runnable tasks (for that CPU) onto the runqueue, which is an **rb-tree** (a type of self-balancing binary search tree), in such a manner that the task that has spent the least amount of time on the processor is the leftmost leaf node on the tree, with succeeding nodes to the right representing the next task to run, then the one after that.

- The **vruntime** value as the key via which tasks are enqueued onto the rb-tree!



Process Management - CFS

- When the kernel needs to schedule, and it asks CFS, the CFS class code we've already mentioned it, the **pick_next_task_fair()** function - simply picks the leftmost leaf node on the tree, returning the pointer to the task structure embedded there; it's, by definition, the task with the lowest vruntime value, effectively, the one that has run the least! (Traversing a tree is a O(log n) time-complexity algorithm, but due to some code optimization and a clever caching of the leftmost leaf node in effect render it into a very desirable O(1) algorithm!).

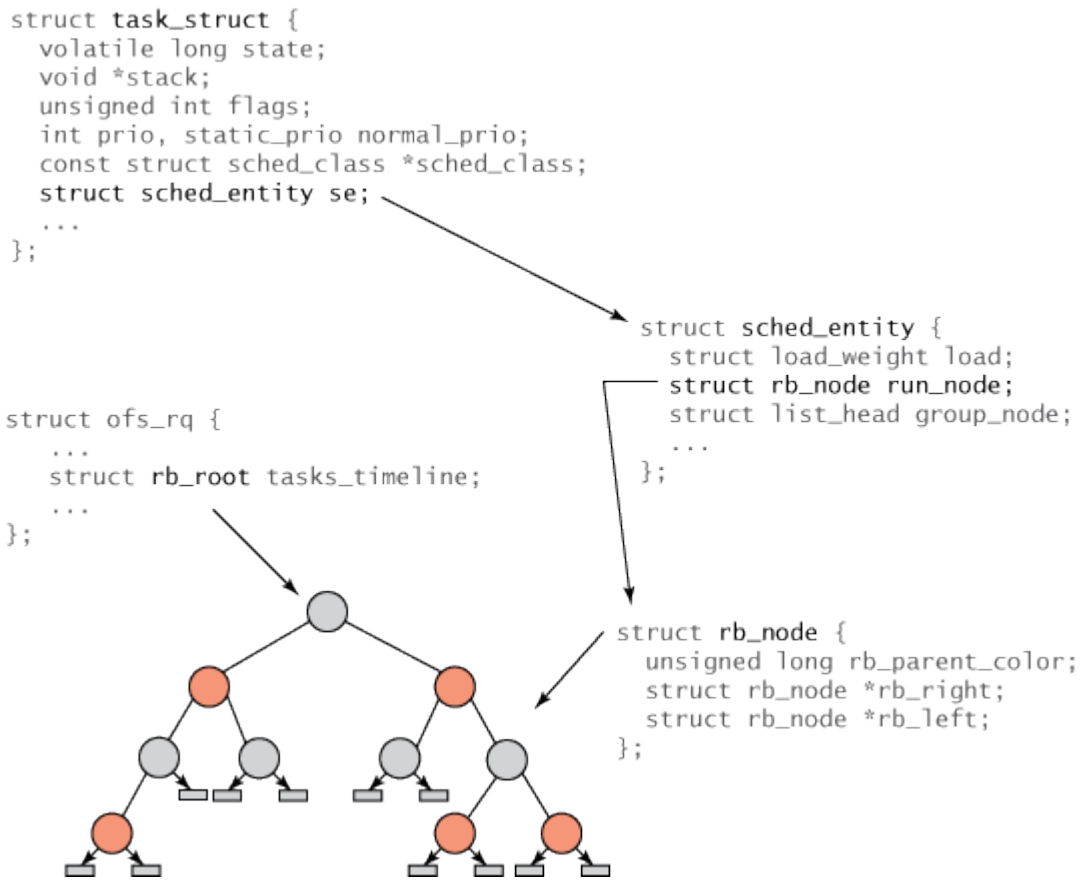
- With tasks (represented by sched_entity objects) stored in the time-ordered red-black tree, tasks with the gravest need for the processor (lowest virtual runtime) are stored toward the left side of the tree, and tasks with the least need of the processor (highest virtual runtimes) are stored toward the right side of the tree. The scheduler then, to be fair, picks the left-most node of the red-black tree to schedule next to maintain fairness. The task accounts for its time with the CPU by adding its execution time to the virtual runtime and is then inserted back into the tree if runnable. In this way, tasks on the left side of the tree are given time to execute, and the contents of the tree migrate from the right to the left to maintain fairness. Therefore, each runnable task chases the other to maintain a balance of execution across the set of runnable tasks.

Process Management - CFS

- All runnable process can be viewed using command **ps ax , ps –A , ps –LA**
- The policy and scheduling priority can be obtained using command **chrt –p <pid of process>**
- Get All schedular information using **chrt –m**

Process Management - CFS

- All tasks within Linux are represented by a task structure called task_struct. This structure (along with others associated with it) fully describes the task and includes the task's current state, its stack, process flags, priority (both static and dynamic), and much more. You can find this and many of the related structures in ./linux/include/linux/sched.h. But because not all tasks are runnable, you won't find any CFS-related fields in task_struct. Instead, a new structure called sched_entity was created to track scheduling information.



Process Management - CFS

- The root of the tree is referenced via the **rb_root** element from the cfs_rq structure (in ./kernel/sched.c). Leaves in a red-black tree contain no information, but internal nodes represent one or more tasks that are runnable. Each node in the red-black tree is represented by an **rb_node**, which contains nothing more than the child references and the color of the parent. The rb_node is contained within the sched_entity structure, which includes the rb_node reference, load weight, and a variety of statistics data. Most importantly, the sched_entity contains the vruntime (64-bit field), which indicates the amount of time the task has run and serves as the index for the red-black tree. Finally, the task_struct sits at the top, which fully describes the task and includes the sched_entity structure.

- The scheduling function is quite simple when it comes to the CFS portion. In ./kernel/sched.c, you'll find the generic schedule() function, which preempts the currently running task (unless it preempts itself with yield()). Note that CFS has no real notion of time slices for preemption, because the preemption time is variable. The currently running task (now preempted) is returned to the red-black tree through a call to put_prev_task (via the scheduling class). When the schedule function comes to identifying the next task to schedule, it calls the pick_next_task function. This function is also generic (within ./kernel/sched.c), but it calls the CFS scheduler through the scheduler class. The pick_next_task function in CFS can be found in ./kernel/sched_fair.c (called pick_next_task_fair()). This function simply picks the left-most task from the red-black tree and returns the associated sched_entity. With this reference, a simple call to **task_of()** identifies the task_struct reference returned. The generic scheduler finally provides the processor to this task.

Process Management - CFS

Priorities and CFS

- CFS doesn't use priorities directly but instead uses them as a decay factor for the time a task is permitted to execute. Lower-priority tasks have higher factors of decay, where higher-priority tasks have lower factors of delay. This means that the time a task is permitted to execute dissipates more quickly for a lower-priority task than for a higher-priority task. That's an elegant solution to avoid maintaining run queues per priority.

Nice values: priorities in the CFS

nice	-20	-19	-18	-17	-16	-15	-14	-13	-12	-11
weight	88761	71755	56483	46273	36291	29154	23254	18705	14949	11916
nice	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
weight	9548	7620	6100	4904	3906	3121	2501	1991	1586	1277
nice	0	1	2	3	4	5	6	7	8	9
weight	1024	820	655	526	423	335	272	215	172	137
nice	10	11	12	13	14	15	16	17	18	19
weight	110	87	70	56	45	36	29	23	18	15

Process Management - CFS

- The priority managed by the CFS is called the nice value in particular, which ranges between -20 and 19. Lower values mean higher priorities (i.e., -20 for the highest priority and 19 for the lowest priority). The default nice value is 0 unless otherwise inherited from a parent process. Each nice value has a corresponding weight value. Note that there is an inverse relationship between weight values and nice values. The weight value determines how large proportion of CPU time a thread gets compared to other threads.

Time slice

- The CFS sets what is called a time slice that is an interval for which a thread is allowed to run without being preempted. The time slice for a thread is proportional to the weight of the thread divided by the total weight of all threads in a run queue. Therefore, the thread that has a relatively high priority is likely to run longer than the other ready threads.

- The function __scheduler_tick that is periodically called by a timer interrupt invokes task_tick of the scheduling class of the current thread. In the CFS, this function pointer eventually executes the function check_preempt_tick().

Process Management - CFS

```
static void check_preempt_tick(struct cfs_rq *cfs_rq, struct sched_entity *curr)
{
    ...
    ideal_runtime = sched_slice(cfs_rq, curr);
    delta_exec = curr->sum_exec_runtime - curr->prev_sum_exec_runtime;
    if (delta_exec > ideal_runtime) {
        resched_task(rq_of(cfs_rq)->curr);
        ...
        return;
    }
    ...
}
```

- The check_preempt_tick() is responsible for checking if the current thread is running any longer than its time slice. If that is the case, check_preempt_tick() calls resched_task() that marks that __schedule() should be executed now to change the running thread. The function sched_slice() is the one that calculates the time slice for the currently running thread.

Process Management - CFS

```
static u64 sched_slice(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    u64 slice = __sched_period(cfs_rq->nr_running + !se->on_rq);
    ...
    cfs_rq = cfs_rq_of(se);
    load = &cfs_rq->load;
    ...
    slice = calc_delta_mine(slice, se->load.weight, load);
    ...
    return slice;
}
```

- The Linux kernel sets a scheduling period during which all ready threads are guaranteed to run at least once. The function __sched_period updates the scheduling period considering the number of ready threads.

Process Management - CFS

- Time accounting in the CFS is done by using the so-called virtual runtime. For a given thread, its virtual runtime is defined as follows:

$$\text{virtual runtime} = (\text{actual runtime}) * 1024/\text{weight}.$$

- Since a weight is proportional to a priority, the virtual runtime of a high priority thread goes slower than that of a low priority thread, when the actual runtime is the same. Note that in the above, 1024 is the weight value for the nice 0 . Thus, the virtual runtime for the thread of nice 0 is equal to its actual runtime.

- All updates to the virtual runtime are performed in update_curr().

Process Management - CFS

```
static void update_curr(struct cfs_rq *cfs_rq)
{
    struct sched_entity *curr = cfs_rq->curr;
    u64 now = rq_of(cfs_rq)->clock_task;
    unsigned long delta_exec;
    ...
    delta_exec = (unsigned long)(now - curr->exec_start);
    ...
    __update_curr(cfs_rq, curr, delta_exec);
    curr->exec_start = now;
    ...
}
```

Process Management - RT

Sched class: Real-time (RT)

- POSIX “real-time” tasks.
- Task priorities: 0 - 99.
- Inverted priorities: 0 highest in kernel, lowest in user space.
- Scheduling policies for tasks at same priority:
  - SCHED_FIFO
  - SCHED_RR, 100ms timeslice by default.
- Implemented with Linked lists.
- Used for short latency sensitive tasks, eg. IRQ threads.

Process Management - RT

- Whenever two processes with the same absolute priority are ready to run, the kernel has a decision to make, because only one can run at a time. If the processes have absolute priority 0, the kernel makes this decision as described in Traditional Scheduling. Otherwise, the decision is as described in this section.

- If two processes are ready to run but have different absolute priorities, the decision is much simpler, and is described in Absolute Priority.

- Each process has a scheduling policy. For processes with absolute priority other than zero, there are two available:
  - First Come First Served
  - Round Robin

- The most sensible case is where all the processes with a certain absolute priority have the same scheduling policy. We'll discuss that first.

Process Management - RT

- In Round Robin, processes share the CPU, each one running for a small quantum of time (“time slice”) and then yielding to another in a circular fashion. Of course, only processes that are ready to run and have the same absolute priority are in this circle.

- In First Come First Served, the process that has been waiting the longest to run gets the CPU, and it keeps it until it voluntarily relinquishes the CPU, runs out of things to do (blocks), or gets preempted by a higher priority process.

- First Come First Served, along with maximal absolute priority and careful control of interrupts and page faults, is the one to use when a process absolutely, positively has to run at full CPU speed or not at all.

- Judicious use of sched_yield function invocations by processes with First Come First Served scheduling policy forms a good compromise between Round Robin and First Come First Served.

Process Management - RT

- To understand how scheduling works when processes of different scheduling policies occupy the same absolute priority, you have to know the nitty gritty details of how processes enter and exit the ready to run list.

- In both cases, the ready to run list is organized as a true queue, where a process gets pushed onto the tail when it becomes ready to run and is popped off the head when the scheduler decides to run it. Note that ready to run and running are two mutually exclusive states. When the scheduler runs a process, that process is no longer ready to run and no longer in the ready to run list. When the process stops running, it may go back to being ready to run again.

- The only difference between a process that is assigned the Round Robin scheduling policy and a process that is assigned First Come First Serve is that in the former case, the process is automatically booted off the CPU after a certain amount of time. When that happens, the process goes back to being ready to run, which means it enters the queue at the tail. The time quantum we're talking about is small. Really small. This is not your father's timesharing. For example, with the Linux kernel, the round robin time slice is a thousand times shorter than its typical time slice for traditional scheduling.

Process Management - RT

- A process begins life with the same scheduling policy as its parent process. Functions described in Basic Scheduling Functions can change it.

- Only a privileged process can set the scheduling policy of a process that has absolute priority higher than 0.

- The RT scheduler's run queue, represented by struct rt_rq, is mainly implemented with an array, each element of which is the head of a linked list that manages the threads of a particular priority.

```
struct rt_prio_array {
    DECLARE_BITMAP(bitmap, MAX_RT_PRIO+1);
    struct list_head queue[MAX_RT_PRIO];
};

struct rt_rq {
    struct rt_prio_array active;
    ...
    int curr;
    ...
}
```

Process Management - RT

- All real-time threads whose priority is x are inserted into a linked list headed by active.queue[x]. When there exists at least one thread in active.queue[x], the x-th bit of active.bitmap is set.

- A newly queued thread is always placed at the end of each list of a corresponding priority in the run queue. The first task on the list of the highest priority available is taken out to run.

- There are two scheduling polices applied for the RT scheduler, which are SCHED_FIFO and SCHED_RR. The difference between the two becomes distinct in task_tick_rt().

Process Management - RT

```c
static void task_tick_rt(struct rq *rq, struct task_struct *p, int queued)
{
    struct sched_rt_entity *rt_se = &p->rt;

    update_curr_rt(rq);

    watchdog(rq, p);

    /*
     * RR tasks need a special form of timeslice management.
     * FIFO tasks have no timeslices.
     */
    if (p->policy != SCHED_RR)
        return;

    if (--p->rt.time_slice)
        return;

    p->rt.time_slice = sched_rr_timeslice;

    /*
     * Requeue to the end of queue if we (and all of our ancestors) are the
     * only element on the queue
     */
    for_each_sched_rt_entity(rt_se) {
        if (rt_se->run_list.prev != rt_se->run_list.next) {
            requeue_task_rt(rq, p, 0);
            set_tsk_need_resched(p);
            return;
        }
    }
}
```

Process Management - RT

- The threads with SCHED_FIFO can run until they stop or yield. There is nothing to be done every tick interrupt. The SCHED_RR threads are given a time slice, which is decremented by 1 on the tick interrupt. When this time slice becomes zero, SCHED_RR threads are enqueued again.

Process Management - STOP

Sched class: STOP

- Highest priority class.
- Only available for SMP (stop_machine() isn't used in UP).
- Can preempt everything and is preempted by nothing.
- Mechanism to stop running everything else and run a specific function on CPU(s).
- No scheduling policies.
- One kernel thread per CPU: "migration/N".
- Used by task migration, CPU hotplug, RCU, ftrace, clockevents, etc.

Process Management - STOP

- STOP schedular class is used to stop other CPUs.
- STOP Schedular Class is useful only on SMP system, if there is no SMP system this schedular class is not required.
- The process whose scheduler class is stop_sched_class is the highest priority process in the system, followed by dl_sched_class and rt_sched_class

- stop_sched_class is used to stop the CPU. It is generally used on SMP systems to achieve load balancing and CPU hot swap. This class has the highest scheduling priority. If your system does not define CONFIG_SMP. You can try to remove this class .

- The stop scheduler class implements the Unix stop_machine feature (depending on the UNIX style, it may be equivalent to other features) ready to splice new code.

- stop_machine is a communication signal: in the case of SMP, it is equivalent to temporarily stopping the operation of other CPUs. It allows one CPU to continue to run, while leaving all other CPUs idle. In the case of a single CPU, this thing is equivalent to turning off the interrupt

Process Management - STOP

- If part shared by multiple CPU need to be modified, and the OS lock, interrupt shutdown and other strategies cannot be used to achieve this function, stop_machine is required.
- The stop scheduler class is the scheduler class with the highest priority, **kernel/sched/stop_task.c**,

```
/*
 * Simple, special scheduling class for the per-CPU stop tasks:
 */
const struct sched_class stop_sched_class = {
        .next                   = &dl_sched_class,

        .enqueue_task           = enqueue_task_stop,
        .dequeue_task           = dequeue_task_stop,
        .yield_task             = yield_task_stop,

        .check_preempt_curr     = check_preempt_curr_stop,

        .pick_next_task         = pick_next_task_stop,
        .put_prev_task          = put_prev_task_stop,
        .set_next_task          = set_next_task_stop,

#ifdef CONFIG_SMP
        .balance                = balance_stop,
        .select_task_rq         = select_task_rq_stop,
        .set_cpus_allowed       = set_cpus_allowed_common,
#endif

        .task_tick              = task_tick_stop,

        .get_rr_interval        = get_rr_interval_stop,

        .prio_changed           = prio_changed_stop,
        .switched_to            = switched_to_stop,
        .update_curr            = update_curr_stop,
};
```

Process Management - STOP

- The kernel provides the sched_set_stop_task function to set the scheduler class of a process stop to stop_sched_class, which is defined in **/kernel/sched/core.c**.

```
void sched_set_stop_task(int cpu, struct task_struct *stop)
{
        struct sched_param param = { .sched_priority = MAX_RT_PRIO - 1 };
        struct task_struct *old_stop = cpu_rq(cpu)->stop;

        if (stop) {
                /*
                 * Make it appear like a SCHED_FIFO task, its something
                 * userspace knows about and won't get confused about.
                 *
                 * Also, it will make PI more or less work without too
                 * much confusion -- but then, stop work should not
                 * rely on PI working anyway.
                 */
                sched_setscheduler_nocheck(stop, SCHED_FIFO, &param);

                stop->sched_class = &stop_sched_class;
        }

        cpu_rq(cpu)->stop = stop;

        if (old_stop) {
                /*
                 * Reset it back to a normal scheduling class so that
                 * it can die in pieces.
                 */
                old_stop->sched_class = &rt_sched_class;
        }
}
```

Process Management - STOP

- sched_set_stop_task binds the stop process to the stop process on the processor numbered cpu, and the scheduling policy of the process is set to SCHED_FIFO, but the scheduler class of the process to which it belongs is set to stop_sched_class, so that when the scheduling class of the process is restored, you only need to The scheduler class of the process can be set to rt_sched_class.

- Few places in the kernel use stop_sched_class, because this scheduler class does not directly schedule processes like dl_shced_class, rt_sched_class and fair_sched_class

- Instead, it is used to complete the stop_machine mechanism, and the implementation of the stop_machine mechanism is in **include/linux/stop_machine.h** and **kernel/stop_machine.c**

```
static struct smp_hotplug_thread cpu_stop_threads = {
        .store                  = &cpu_stopper.thread,
        .thread_should_run      = cpu_stop_should_run,
        .thread_fn              = cpu_stopper_thread,
        .thread_comm            = "migration/%u",
        .create                 = cpu_stop_create,
        .park                   = cpu_stop_park,
        .selfparking            = true,
};
```

Process Management - STOP

Generally speaking, the kernel will use stop_machine technology in the following situations

application	description
module install and remove	Add delete modules, load and delete modules without restarting the kernel
cpu hotplug	CPU hot plug, used to perform task migration work, cpu_stop_threads , the task is completed by the migration kernel thread bound to the CPU
memory hotplug	Memory hot swap
ftrace	Kernel trace and debug functions, see kernel/trace/ftrace.c
hwlat_detector	Check the latency and debug functions introduced by the system hardware
Kernel Hotpatch	Ksplice can dynamically apply kernel patches in less than a second, without rebooting

Process Management - DEADLINE

Sched class: Deadline (DL)

- Introduced by Dario Faggioli & Juri Lelli, v3.14 (2013).
- Highest priority tasks in the system.
- Scheduling policy: SCHED_DEADLINE.
- Implemented with red-black tree (self balancing).
- Used for periodic real time tasks, eg. Video encoding/decoding.

Process Management - DEADLINE

- Used for tasks that must finish before a deadline. Audio processing and video encoding/decoding are among the typical use cases a developer would want to use this scheduling class for. DL has the highest priority in a Uni Processor machine.

- The SCHED_DEADLINE policy contained inside the sched_dl scheduling class is basically an implementation of the Earliest Deadline First (EDF) scheduling algorithm, augmented with a mechanism (called Constant Bandwidth Server, CBS) that makes it possible to isolate the behavior of tasks between each other.

Process Management - IDLE

Sched class: Idle

- Lowest priority scheduling class.
- No scheduling policies.
- One kernel thread (idle) per CPU: “swapper/N”.
- Idle thread runs only when nothing else is runnable on a CPU.
- Idle thread may take the CPU to low power state.

Process Management - Preemption

- Let's take a hypothetical situation: you're running on a system with one CPU. An analog clock app is running on the GUI along with a C program, a.out, whose one line of code is while(1);. So, what do you think: will the CPU hogger while 1 process indefinitely hog the CPU, thus causing the GUI clock app to stop ticking (will its second hand stop moving altogether)?

- A little thought (and experimentation) will reveal that, indeed, the GUI clock app keeps ticking in spite of the naughty CPU hogger app! Actually, this is really the whole point of having an OS-level scheduler: it can, and does, preempt the CPU-hogging user space process. (We briefly discussed the CFS algorithm previously; CFS will cause the aggressive CPU hogger process to accumulate a huge vruntime value and thus move more to the right on its rb-tree runqueue, thus penalizing itself!) All modern OSes support this type of preemption – it's called **user-mode preemption.**

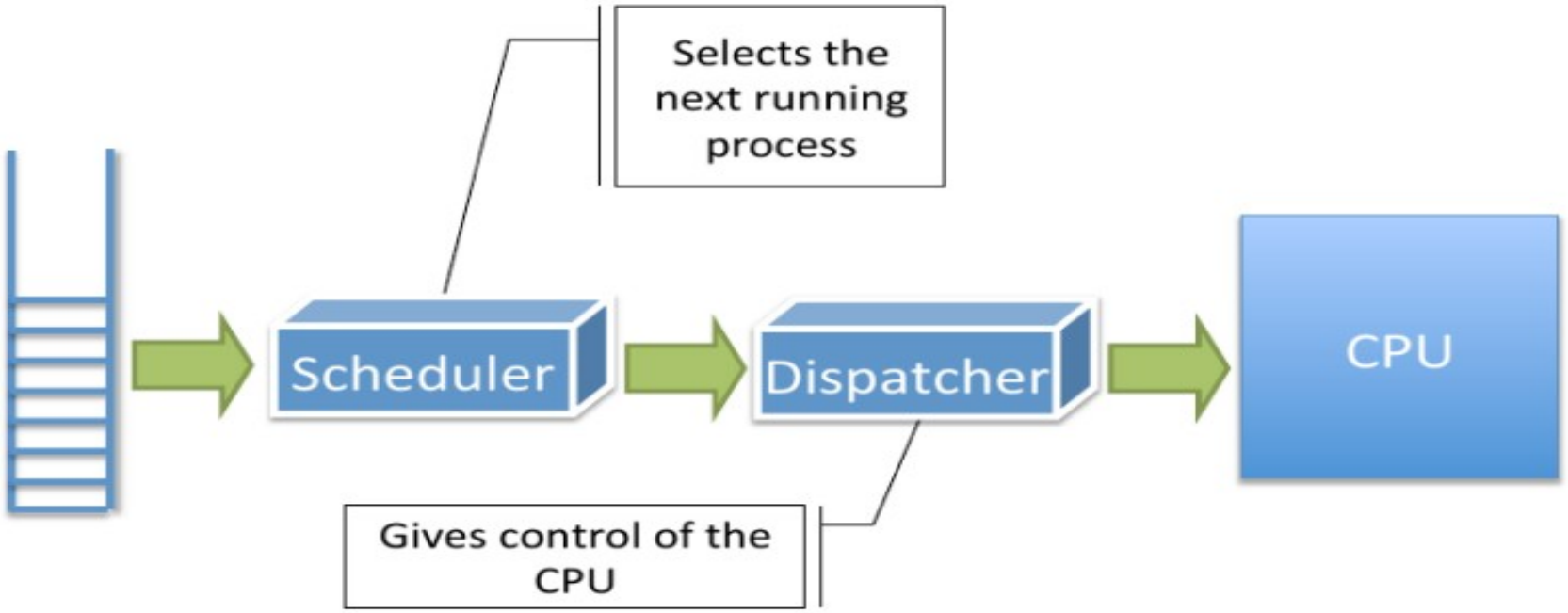
- But now, consider this: what if you write a kernel module that performs the same while(1) infinite loop on a single processor system? This could be a problem: the system will now simply hang. How will the OS preempt itself (as we understand that kernel modules run in kernel mode at kernel privilege)? Well, guess what: for many years now, Linux has provided a build-time configuration option to make the kernel preemptible, CONFIG_PREEMPT. Once this CONFIG_PREEMPT kernel config option is turned on and the kernel is built and booted into, we're now running on a preemptible kernel – where the OS has the ability to preempt itself.

Process Management - Preemption

Preemption type	Characteristics	Appropriate for
CONFIG_PREEMPT_NONE	Traditional model, geared toward high overall throughput.	Server/enterprise-class and compute-intensive systems
CONFIG_PREEMPT_VOLUNTARY	Preemptible kernel (desktop); more explicit preemption opportunity points within the OS; leads to lower latencies, better app response. Typically the default for distros.	Workstations/desktops, laptops running Linux for the desktop
CONFIG_PREEMPT	LowLat kernel; (almost) the entire kernel is preemptible; implies involuntary preemption of even kernel code paths is now possible; yields even lower latencies (tens of us to low hundreds us range on average) at the cost of slightly lower throughput and slight runtime overhead.	Fast multimedia systems (desktops, laptops, even modern embedded products: smartphones, tablets, and so on)

Process Management – Context Switch

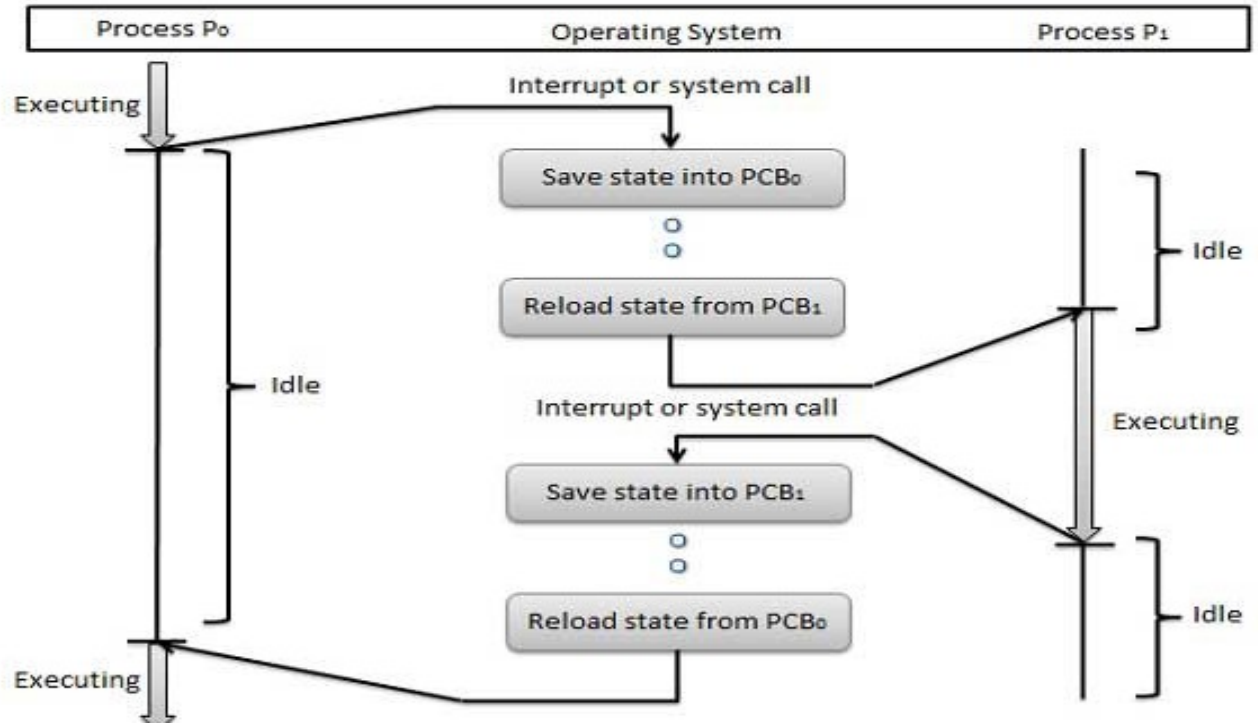
Scheduler and dispatcher



Process Management – Context Switch

Context switching (I)

- The act of switching from one process to another
- The system has to:
  - Save the context of the current process
  - Restore the context of the new process



Process Management – Context Switch

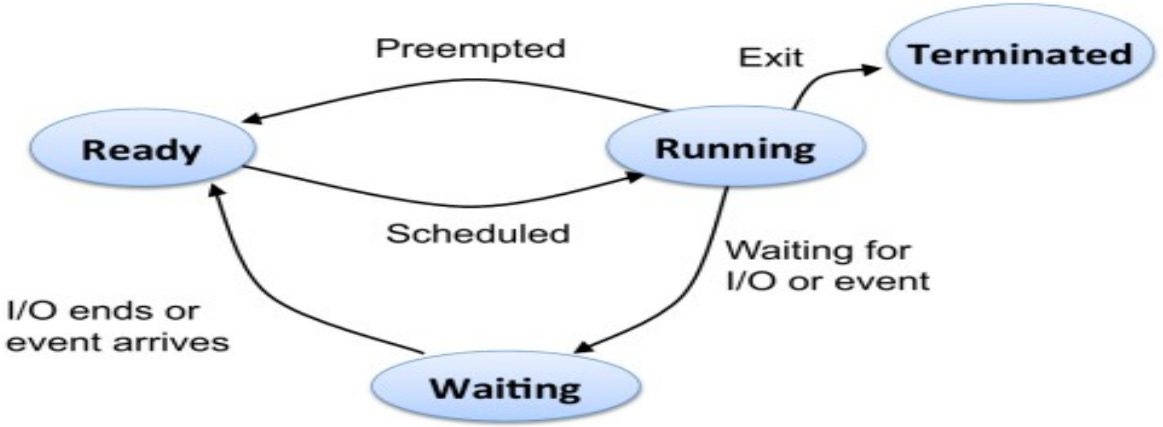
Context switching (II)

- What is the context of a process?
  - Program Counter
  - Stack Pointer
  - Registers
  - Code + Data + Stack (also called Address Space)
  - Other state information maintained by the OS for the process (open files, scheduling info, I/O devices being used, etc.)
- All this information is usually stored in a structure called **Process Control Block** (PCB)

Process Management – process states

Process State

- Multitasking implies process states.
  - Names for these states are not standardised, but they have similar functionality:
    - **Ready**: ready to run, but waiting to be scheduled;
    - **Running**: executing at the moment;
    - **Waiting**: waiting for I/O or an event;
    - **Terminated**: no longer ready to execute.



Process Management – process states

Scheduling events

- A process switches from the **running** state to **waiting** state (e.g. I/O request);
- A process switches from the **running** state to the **ready** state (e.g. time slice expires);
- A process switches from **waiting** state to **ready** state (e.g. completion of an I/O operation);
- A process **terminates**.

Notice

All executing tasks go from **ready** to **running** state. Note that, there is only one task in the **running** state, per CPU.

Kthread

- An executing instance of a program is called a process. Some operating systems use the term ‘task‘ to refer to a program that is being executed. The process is a heavyweight process. The context switch between the process is time-consuming.

- A thread is an independent flow of control that operates within the same address space as other independent flows of control within a process.

- One process can have multiple threads, with each thread executing different code concurrently, while sharing data and synchronizing much more easily than cooperating processes. Threads require fewer system resources than processes and can start more quickly. Threads, also known as lightweight processes.

- One of the advantages of the thread is that since all the threads within the processes share the same address space, the communication between the threads is far easier and less time-consuming as compared to processes. This approach has one disadvantage also. It leads to several concurrency issues and requires synchronization mechanisms to handle the same.

Kthread

- Whenever we are creating a thread, it has to manage by someone. So that management follows like below.
  - A thread is a sequence of instructions.
  - CPU can handle one instruction at a time.
  - To switch between instructions on parallel threads, the execution state needs to be saved.
  - Execution state in its simplest form is a program counter and CPU registers.
  - The program counter tells us what instruction to execute next.
  - CPU registers hold execution arguments, for example, addition operands.
  - This alternation between threads requires management.
  - Management includes saving state, restoring state, deciding what thread to pick next.

Kthread

There are two types of threads.

- **User Level Thread**

  In this type, the kernel is not aware of these threads. Everything is maintained by the user thread library. That thread library contains code for creating and destroying threads, for passing messages and data between threads, for scheduling thread execution, and for saving and restoring thread contexts. So all will be in User Space.

- **Kernel Level Thread**

  Kernel level threads are managed by the OS, therefore, thread operations are implemented in the kernel code. There is no thread management code in the application area.

Kthread

kthread_create

This API creates a kthread.

struct task_struct * kthread_create (int (* threadfn(void *data), void *data, const char namefmt[], ...);

Argument:

threadfn – the function to run until signal_pending(current).

data – data ptr for threadfn.

namefmt[] – printf-style name for the thread.

... – variable arguments

This helper function creates and names a kernel thread. But we need to wake up that thread manually. When woken, the thread will run threadfn() with data as its argument.

Kthread

threadfn can either call do_exit directly if it is a standalone thread for which no one will call kthread_stop, or return when ‘kthread_should_stop‘ is true (which means kthread_stop has been called). The return value should be zero or a negative error number; it will be passed to kthread_stop.

It Returns task_struct or ERR_PTR(-ENOMEM).

Kthread

wake_up_process

This is used to Wake up a specific process.

int wake_up_process (struct task_struct * p);

Argument:

p – The process to be woken up.

Attempt to wake up the nominated process and move it to the set of runnable processes.

It returns 1 if the process was woken up, 0 if it was already running.

It may be assumed that this function implies a write memory barrier before changing the task state if and only if any tasks are woken up.

Kthread

wake_up_process

This is used to Wake up a specific process.

int wake_up_process (struct task_struct * p);

Argument:

p – The process to be woken up.

Attempt to wake up the nominated process and move it to the set of runnable processes.

It returns 1 if the process was woken up, 0 if it was already running.

It may be assumed that this function implies a write memory barrier before changing the task state if and only if any tasks are woken up.

Kthread

kthread_stop

It stops a thread created by kthread_create.

int kthread_stop (struct task_struct *k);

Parameter:

k – thread created by kthread_create.

Sets kthread_should_stop for k to return true, wakes it and waits for it to exit. Your threadfn must not call do_exit itself, if you use this function! This can also be called after kthread_create instead of calling wake_up_process: the thread will exit without calling threadfn.

It Returns the result of threadfn, or –EINTR if wake_up_process was never called.

Kthread

kthread_should_stop

should this kthread return now.

int kthread_should_stop (void);

When someone calls kthread_stop on your kthread, it will be woken and this will return true. You should then return, and your return value will be passed through to kthread_stop.

Kthread

kthread_bind

This is used to bind a just-created kthread to a CPU.

void kthread_bind (struct task_struct *k, unsigned int cpu);

Argument:

k – thread created by kthread_create.

cpu – CPU (might not be online, must be possible) for k to run on.

Kthread

First, we have to create our thread that has the argument of void * and should return int value. We should follow some conditions in our thread function. It is advisable.

If that thread is a long run thread, we need to check kthread_should_stop() every time, because any function may call kthread_stop. If any function called kthread_stop, that time kthread_should_stop will return true. We have to exit our thread function if true value been returned by kthread_should_stop.

But if your thread function is not running long, then let that thread finish its task and kill itself using do_exit.

In my thread function, let's print something every minute and it is a continuous process. So let's check the kthread_should_stop every time. See the below snippet to understand.

Kthread

```
int thread_function(void *pv)
{
 int i=0;
 while(!kthread_should_stop())
 {
   printk(KERN_INFO "In Thread Function %d\n", i++);
   msleep(1000);
 }
 return 0;
}
```

Kthread

So as of now, we have our thread function to run. Now, we will create a kernel thread using kthread_create and start the kernel thread using wake_up_process.

```
static struct task_struct *thread1;
thread1  = kthread_create(thread_function,NULL,"Thread");
if(thread1)
{
 wake_up_process(thread1);
}
else
{
 printk(KERN_ERR "Cannot create kthread\n");
}
```

Kthread

There is another function that does both processes (create and start). That is kthread_run(). You can replace both kthread_create and wake_up_process using this function.

kthread_run

This is used to create and wake a thread.

kthread_run (threadfn, data, namefmt, ...);

Argument:

threadfn – the function to run until signal_pending(current).

data – data ptr for threadfn.

namefmt – printf-style name for the thread.

... – variable arguments

Convenient wrapper for kthread_create followed by wake_up_process.

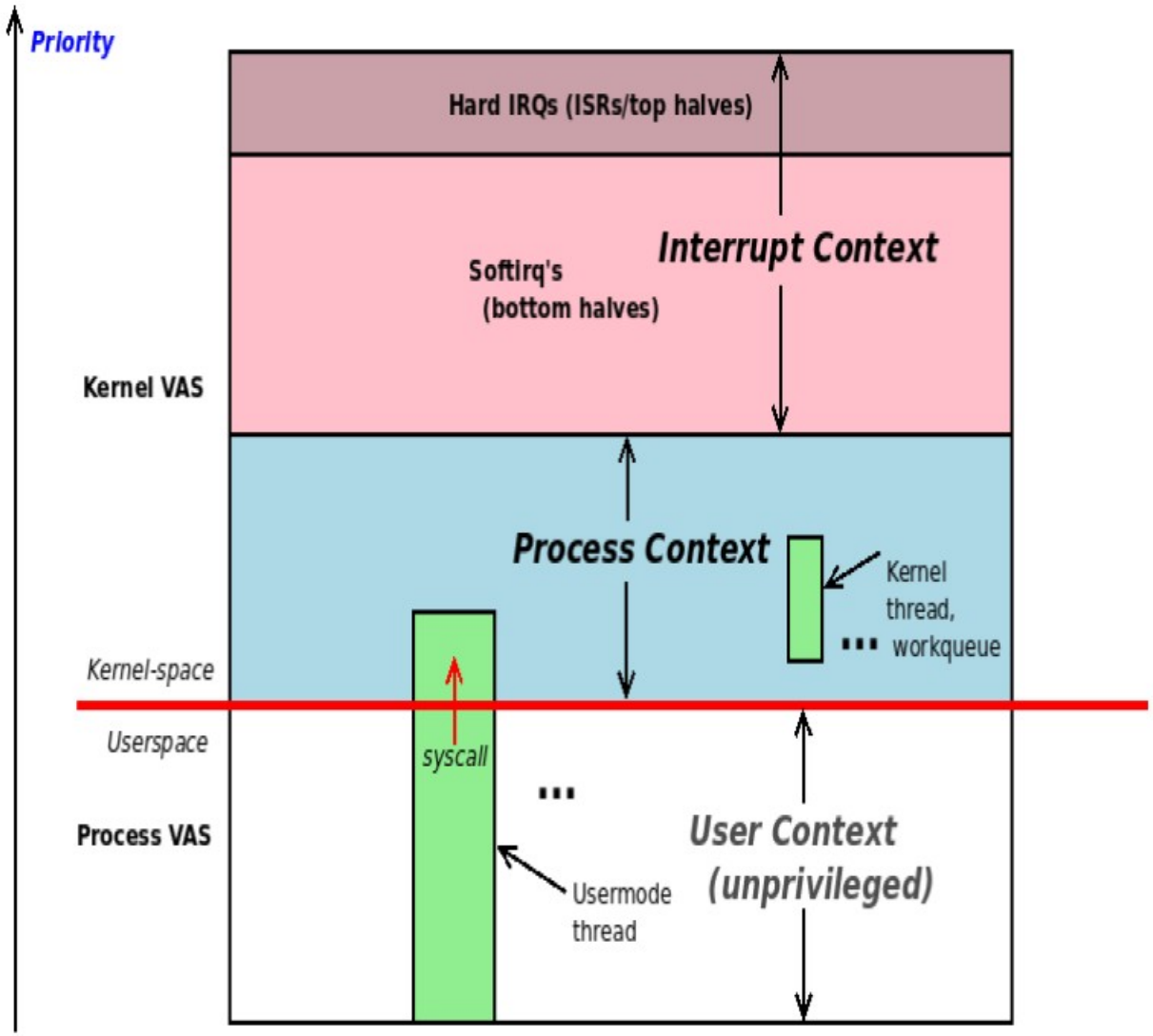
It returns the kthread or ERR_PTR(-ENOMEM).

Kthread

You can see the below snippet which is using kthread_run.

```
static struct task_struct *thread1;
thread1 = kthread_run(thread_function,NULL,"Thread");
if(thread1)
{
 printk(KERN_ERR "Kthread Created Successfully...\n");
}
else
{
 printk(KERN_ERR "Cannot create kthread\n");
}
```

Bottom Halves



Bottom Halves

Splitting the execution of interrupt handlers in 2 parts

- Top half
  - This is the real interrupt handler, which should complete as quickly as possible since all interrupts are disabled. It takes the data out of the device and if substantial post-processing is needed, schedule a bottom half to handle it.
- Bottom half
  - Is the general Linux name for various mechanisms which allow to postpone the handling of interrupt-related work. Implemented in Linux as softirqs, tasklets or workqueues.

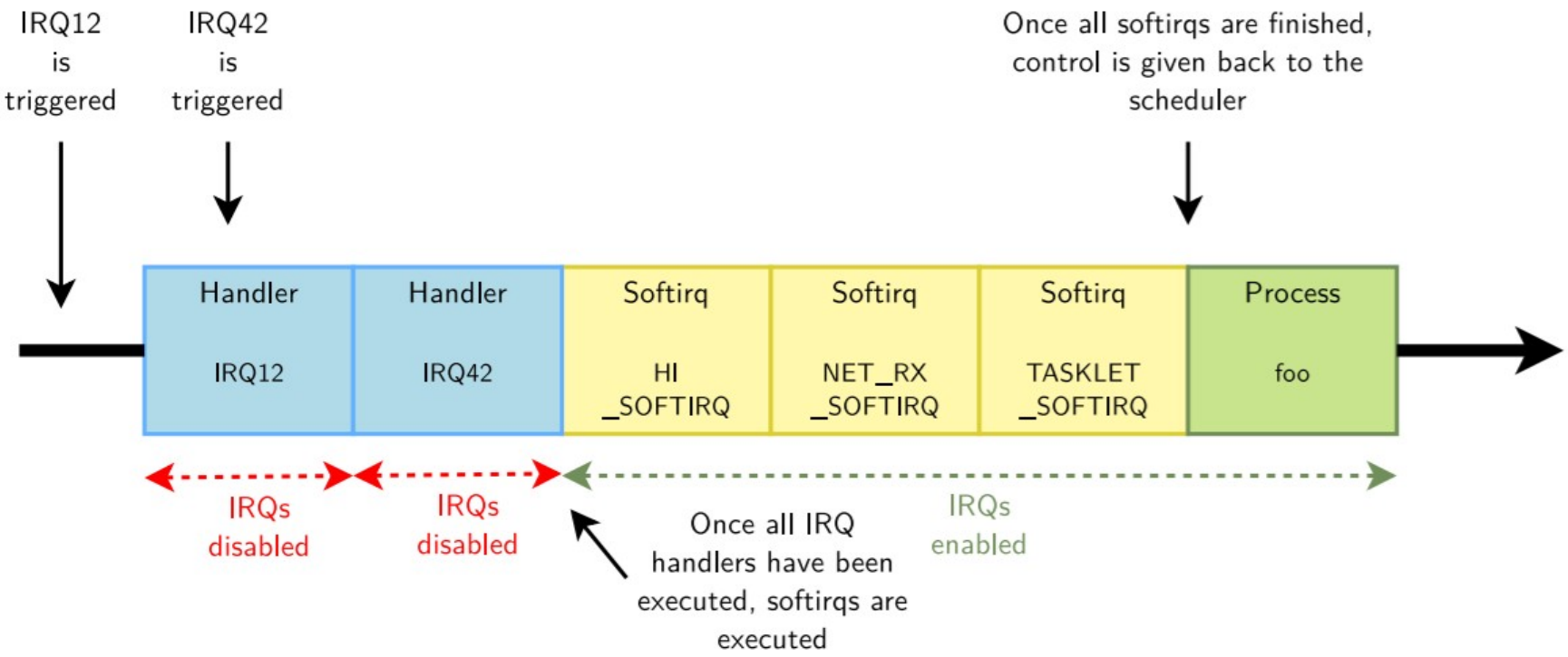
Bottom Halves

- When Interrupt triggers, Interrupt Handler should be executed very quickly and it should not run for more time (it should not perform time-consuming tasks). If we have the interrupt handler which is doing more tasks then we need to divide it into two halves.
  - Top Half
  - Bottom Half

- The top Half is nothing but our interrupt handler. If we want to do less work, then the top half is more than enough. No need for the bottom half in that situation. But if we have more work when interrupt hits, then we need the bottom half. The bottom half runs in the future, at a more convenient time, with all interrupts enabled. So, The job of bottom halves is to perform any interrupt-related work not performed by the interrupt handler.

- There are 4 bottom half mechanisms are available in Linux:
  - Workqueue in Linux Kernel – Executed in a process context.
  - Threaded IRQ
  - Softirq – Executed in an atomic context.
  - Tasklet – Executed in an atomic context.

Bottom Halves



Bottom Halves

- Softirqs are a form of bottom half processing
- The softirqs handlers are executed with all interrupts enabled, and a given softirq handler can run simultaneously on multiple CPUs
- They are executed once all interrupt handlers have completed, before the kernel resumes scheduling processes, so sleeping is not allowed.
- The number of softirqs is fixed in the system, so softirqs are not directly used by drivers, but by complete kernel subsystems (network, etc.)
- The list of softirqs is defined in `include/linux/interrupt.h`: `HI_SOFTIRQ`, `TIMER_SOFTIRQ`, `NET_TX_SOFTIRQ`, `NET_RX_SOFTIRQ`, `BLOCK_SOFTIRQ`, `IRQ_POLL_SOFTIRQ`, `TASKLET_SOFTIRQ`, `SCHED_SOFTIRQ`, `HRTIMER_SOFTIRQ`, `RCU_SOFTIRQ`
- `HI_SOFTIRQ` and `TASKLET_SOFTIRQ` are used to execute tasklets

Thank you